EXPERIMENT NO. 9
Object Detection and Recognition using Deep Learning Models

```
!pip -q install ultralytics
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import glob
import shutil
import time
from collections import Counter
from ultralytics import YOLO

# 1. CREATE RESULT FOLDER
RESULT_DIR = "/content/Experiment_09_Results"
if os.path.exists(RESULT_DIR):
    shutil.rmtree(RESULT_DIR)
os.makedirs(RESULT_DIR)
print("=" * 75)
print("EXPERIMENT NO. 9")
print("Object Detection and Recognition using Deep Learning Models")
print("=" * 75)

# 2. FIND IMAGE ALREADY UPLOADED TO /content
print("\n🔍 Searching for uploaded image...")
valid_extensions = (".jpg", ".jpeg", ".png", ".webp")
image_files = [
    f for f in glob.glob("/content/*")
    if f.lower().endswith(valid_extensions)
    and not f.startswith(RESULT_DIR)
]
if len(image_files) == 0:
    raise FileNotFoundError(
        "❌ No image found in /content.\n"
        "Please upload your image using Colab Files panel:\n"
        "📂 Files → Upload → Select your image → Run this cell again."
    )
# Select first available image
image_path = image_files[0]
print("✅ Image found:", os.path.basename(image_path))

# 3. READ IMAGE
image = cv2.imread(image_path)
if image is None:
    raise ValueError("❌ Unable to read the selected image.")
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
height, width = image.shape[:2]
channels = image.shape[2] if len(image.shape) == 3 else 1
print("\n📌 IMAGE INFORMATION")
```

```
print("-" * 50)
print("File Name      :", os.path.basename(image_path))
print("Width          :", width, "pixels")
print("Height         :", height, "pixels")
print("Channels       :", channels)
print("Total Pixels   :", width * height)
print("Image Size     :", round(os.path.getsize(image_path) / 1024, 2), "KB")

# 4. DISPLAY ORIGINAL IMAGE
plt.figure(figsize=(10, 7))
plt.imshow(image_rgb)
plt.title("Original Input Image")
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(RESULT_DIR, "01_Original_Image.png"),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 5. LOAD PRE-TRAINED YOLO MODEL
print("\n🤖 Loading Pre-trained YOLO11n Model...")
model = YOLO("yolo11n.pt")
print("✅ YOLO model loaded successfully.")

# 6. OBJECT DETECTION
# print("\n🔎 Performing Object Detection...")
start_time = time.time()
results = model.predict(
    source=image,
    imgsz=640,
    conf=0.25,
    iou=0.70,
    verbose=False
)
inference_time = time.time() - start_time
result = results[0]
print("✅ Detection completed.")
print("Inference Time:", round(inference_time, 4), "seconds")

# 7. EXTRACT DETECTION RESULTS
detections = []
```

```
if result.boxes is not None and len(result.boxes) > 0:
    boxes = result.boxes.xyxy.cpu().numpy()
    confidences = result.boxes.conf.cpu().numpy()
 class_ids = result.boxes.cls.cpu().numpy().astype(int)
  for box, confidence, class_id in zip(
        boxes, confidences, class_ids
    ):
        x1, y1, x2, y2 = box
        class_name = model.names[class_id]
        detections.append({
            "Object": class_name,
            "Confidence": round(float(confidence), 4),
            "X1": int(x1),
            "Y1": int(y1),
            "X2": int(x2),
            "Y2": int(y2),
            "Width": int(x2 - x1),
            "Height": int(y2 - y1)
        })

# 8. DETECTION TABLE
df = pd.DataFrame(detections)
print("\n📋 DETECTION RESULTS")
print("=" * 75)
if len(df) > 0:
    print(df.to_string(index=False))
else:
    print("⚠️ No objects detected.")
df.to_csv(
    os.path.join(RESULT_DIR, "02_Detection_Results.csv"),
    index=False
)

# 9. CREATE ANNOTATED IMAGE
annotated = result.plot()
annotated_rgb = cv2.cvtColor(
    annotated,
    cv2.COLOR_BGR2RGB
)
cv2.imwrite(
    os.path.join(RESULT_DIR, "03_Detected_Objects.jpg"),
    annotated
)
plt.figure(figsize=(12, 8))
plt.imshow(annotated_rgb)
plt.title("YOLO Object Detection Result")
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(RESULT_DIR, "04_Object_Detection_Result.png"),
```

```
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 10. OBJECT COUNT
if len(df) > 0:
     object_counts = Counter(df["Object"])
     count_df = pd.DataFrame(
        list(object_counts.items()),
        columns=["Object", "Count"]
    ).sort_values(
        by="Count",
        ascending=False
    )
else:
        count_df = pd.DataFrame(
        columns=["Object", "Count"]
    )

print("\n📊 OBJECT COUNT")
print("-" * 50)
if len(count_df) > 0:
    print(count_df.to_string(index=False))
else:
    print("No objects detected.")
count_df.to_csv(
    os.path.join(RESULT_DIR, "05_Object_Count.csv"),
    index=False
)

# 11. OBJECT COUNT GRAPH
if len(count_df) > 0:
    plt.figure(figsize=(10, 6))
    plt.bar(
        count_df["Object"],
        count_df["Count"]
    )
    plt.xlabel("Object Class")
    plt.ylabel("Number of Objects")
    plt.title("Detected Object Count")
    plt.xticks(
        rotation=45,
        ha="right"
    )
    plt.tight_layout()
    plt.savefig(
        os.path.join(RESULT_DIR, "06_Object_Count_Graph.png"),
        dpi=200,
        bbox_inches="tight"
```

```
    )
    plt.show()

# 12. CONFIDENCE SCORE GRAPH
if len(df) > 0:
 labels = [
        f"{obj}\n{conf:.2f}"
        for obj, conf in zip(
            df["Object"],
            df["Confidence"]
        )
    ]
    plt.figure(figsize=(12, 6))
    plt.bar(
        range(len(df)),
        df["Confidence"]
    )
    plt.xticks(
        range(len(df)),
        labels,
        rotation=45,
        ha="right"
    )
    plt.xlabel("Detected Objects")
    plt.ylabel("Confidence Score")
    plt.title("Confidence Scores of Detected Objects")
    plt.ylim(0, 1)
    plt.tight_layout()
    plt.savefig(
        os.path.join(RESULT_DIR, "07_Confidence_Scores.png"),
        dpi=200,
        bbox_inches="tight"
    )
    plt.show()
# 13. ORIGINAL VS DETECTED COMPARISON
plt.figure(figsize=(16, 7))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(annotated_rgb)
plt.title("Detected Objects")
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(RESULT_DIR, "08_Original_vs_Detected.png"),
    dpi=200,
    bbox_inches="tight"
)
```

```
plt.show()

# 14. DETECTION PERFORMANCE METRICS
total_objects = len(df)
if total_objects > 0:
    average_confidence = df["Confidence"].mean()
    maximum_confidence = df["Confidence"].max()
    minimum_confidence = df["Confidence"].min()
else:
    average_confidence = 0
    maximum_confidence = 0
    minimum_confidence = 0
metrics = pd.DataFrame({
    "Parameter": [
        "Image Width",
        "Image Height",
        "Total Detected Objects",
        "Average Confidence",
        "Maximum Confidence",
        "Minimum Confidence",
        "Inference Time (seconds)",
        "Approx. Inference FPS"
    ],
    "Value": [
        width,
        height,
        total_objects,
        round(average_confidence, 4),
        round(maximum_confidence, 4),
        round(minimum_confidence, 4),
        round(inference_time, 4),
        round(1 / inference_time, 2)
        if inference_time > 0 else 0
    ]
})
print("\n⚡ DETECTION PERFORMANCE")
print("=" * 75)
print(metrics.to_string(index=False))
metrics.to_csv(
    os.path.join(
        RESULT_DIR,
        "09_Detection_Performance_Metrics.csv"
    ),
    index=False
)


# 15. CONFIDENCE DISTRIBUTION
if len(df) > 0:
    plt.figure(figsize=(9, 6))
```

```
    plt.hist(
        df["Confidence"],
        bins=10,
        edgecolor="black"
    )
    plt.xlabel("Confidence Score")
    plt.ylabel("Number of Detections")
    plt.title("Confidence Score Distribution")
    plt.tight_layout()
    plt.savefig(
        os.path.join(
            RESULT_DIR,"10_Confidence_Distribution.png"),dpi=200,bbox_inches="tight")
    plt.show()

# 16. FINAL OBSERVATIONS
print("\n")
print("=" * 75)
print("FINAL OBSERVATIONS")
print("=" * 75)
print(f"""
1. A pre-trained YOLO deep learning model was used for object detection.
2. The input image was successfully processed using the neural network.
3. Objects were localized using bounding boxes.
4. Each detected object was assigned a class label.
5. Confidence scores were calculated for the detected objects.
6. Multiple objects can be detected simultaneously in a single image.
7. Higher confidence scores indicate stronger model confidence in the prediction.
8. The detected-object image provides both localization and classification results.
9. Object detection can be applied in surveillance, autonomous vehicles,
   traffic monitoring, robotics, security and smart-city applications.
10. The model used in this experiment is pre-trained on the COCO object
    detection dataset.
""")
# 17. GENERATED FILES
print("\n📂 GENERATED EXPERIMENT FILES")
print("=" * 75)
for file in sorted(os.listdir(RESULT_DIR)):
    file_path = os.path.join(RESULT_DIR, file)
    size_kb = os.path.getsize(file_path) / 1024
    print(f"✓ {file:<45} {size_kb:.2f} KB")

print("\n" + "=" * 75)
print("✅ EXPERIMENT 9 COMPLETED SUCCESSFULLY")
print("=" * 75)
print("\n📂 Results saved in:")
print(RESULT_DIR)
```

OUTPUT –

```
===========================================================================
==========
EXPERIMENT NO. 9
```

Object Detection and Recognition using Deep Learning Models
==========

🔍 Searching for uploaded image...
✅ Image found: -2.jpg

📌 IMAGE INFORMATION

File Name   : -2.jpg
Width   : 736 pixels
Height   : 1104 pixels
Channels   : 3
Total Pixels   : 812544
Image Size   : 97.42 KB

Original Input Image

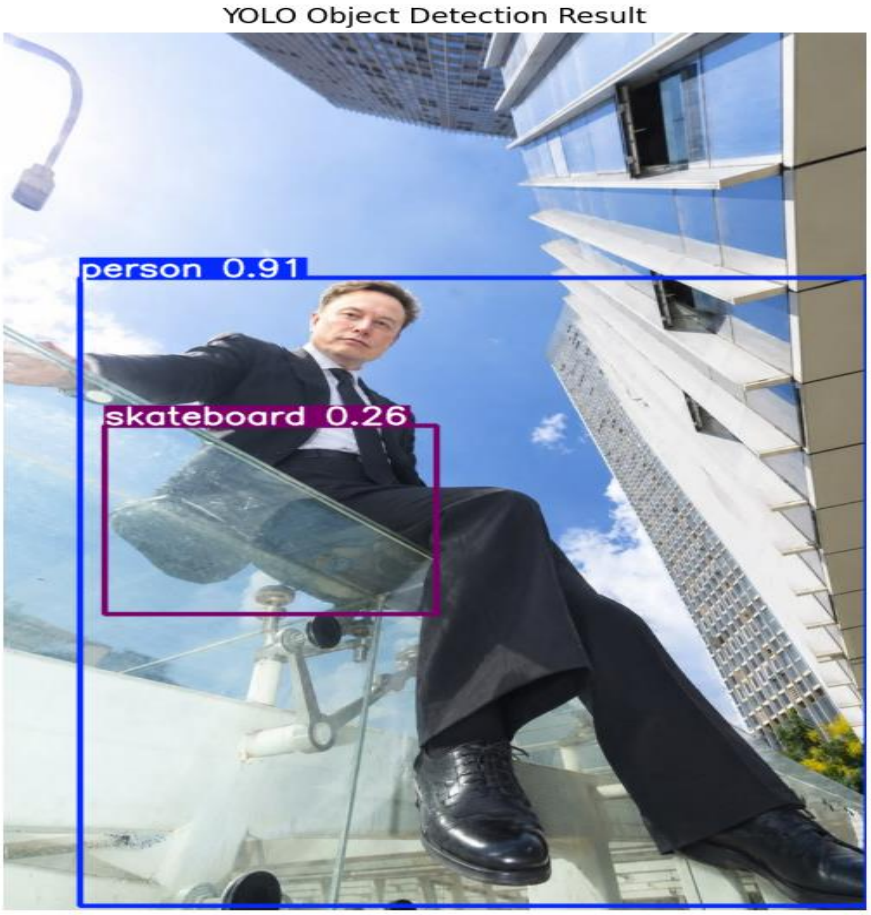


🤖 Loading Pre-trained YOLO11n Model...
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% ━━━━━━━━━━━━━━━━━━━━ 5.4MB 79.1MB/s 0.1s
✅ YOLO model loaded successfully.

🔎 Performing Object Detection...
✅ Detection completed.
Inference Time: 8.8252 seconds

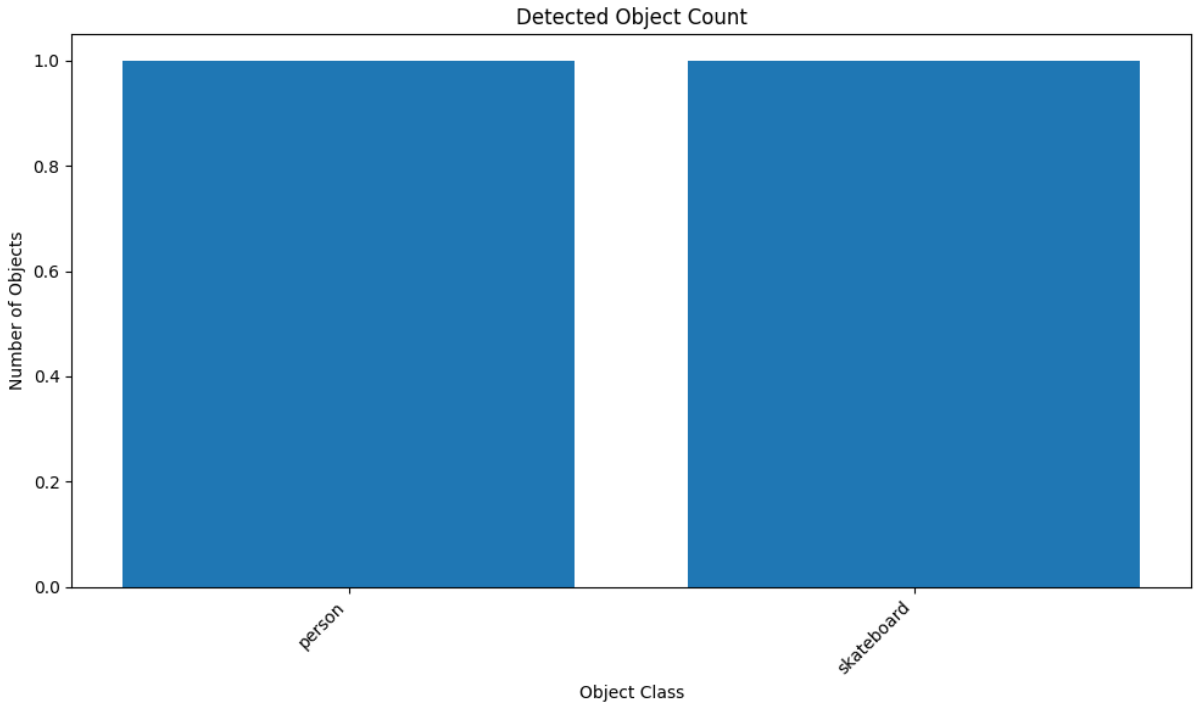
📋 DETECTION RESULTS
==========

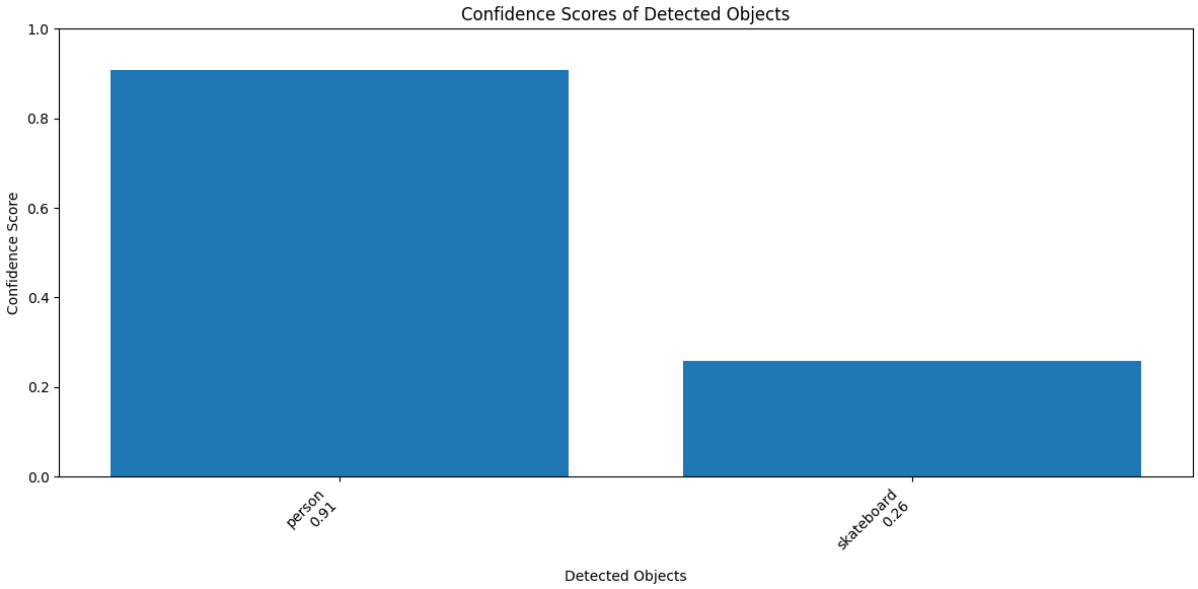
Object	Confidence	X1	Y1	X2	Y2	Width	Height
person	0.9071	65	308	736	1098	670	790
skateboard	0.2574	86	494	369	731	283	236



📊 OBJECT COUNT

Object	Count
person	1
skateboard	1

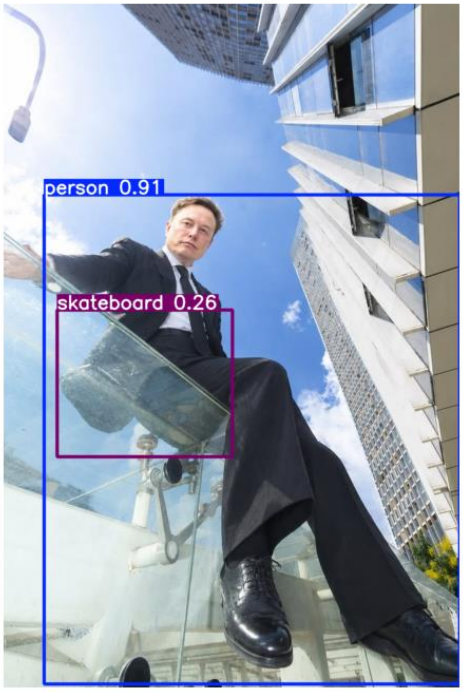

Detected Object Count
Number of Objects
1.0
0.8
0.6
0.4
0.2
0.0
person
skateboard
Object Class


Confidence Scores of Detected Objects
Confidence Score
1.0
0.8
0.6
0.4
0.2
0.0
person
0.91
skateboard
0.26
Detected Objects

Original Image

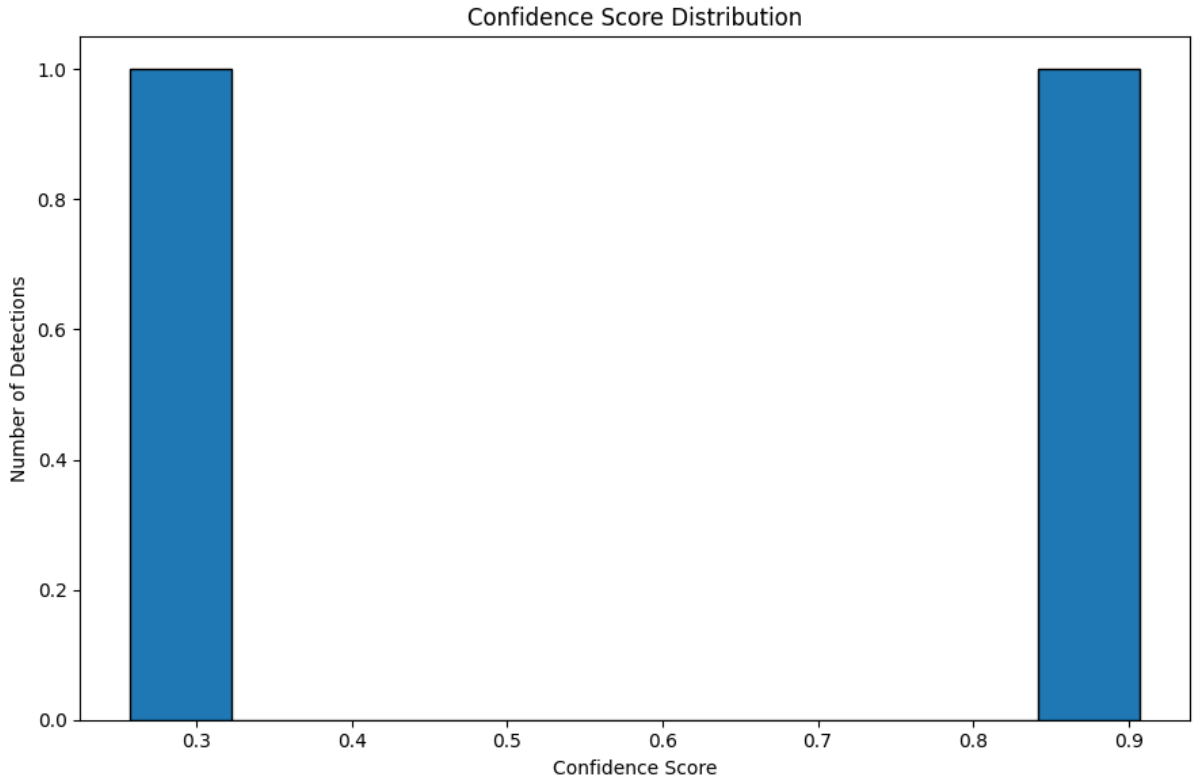


Detected Objects



⚡ DETECTION PERFORMANCE

Parameter	Value
Image Width	736.0000
Image Height	1104.0000
Total Detected Objects	2.0000
Average Confidence	0.5822
Maximum Confidence	0.9071
Minimum Confidence	0.2574
Inference Time (seconds)	8.8252
Approx. Inference FPS	0.1100



==========

FINAL OBSERVATIONS

==========

1. A pre-trained YOLO deep learning model was used for object detection.
2. The input image was successfully processed using the neural network.
3. Objects were localized using bounding boxes.
4. Each detected object was assigned a class label.
5. Confidence scores were calculated for the detected objects.
6. Multiple objects can be detected simultaneously in a single image.
7. Higher confidence scores indicate stronger model confidence in the prediction.
8. The detected-object image provides both localization and classification results.
9. Object detection can be applied in surveillance, autonomous vehicles, traffic monitoring, robotics, security and smart-city applications.
10. The model used in this experiment is pre-trained on the COCO object detection dataset.

📁 GENERATED EXPERIMENT FILES

==========

✓ 01_Original_Image.png	1117.15 KB
✓ 02_Detection_Results.csv	0.12 KB
✓ 03_Detected_Objects.jpg	199.40 KB

✓ 04_Object_Detection_Result.png 1369.91 KB
✓ 05_Object_Count.csv 0.03 KB
✓ 06_Object_Count_Graph.png 51.82 KB
✓ 07_Confidence_Scores.png 63.36 KB
✓ 08_Original_vs_Detected.png 1266.83 KB
✓ 09_Detection_Performance_Metrics.csv 0.21 KB
✓ 10_Confidence_Distribution.png 47.13 KB

==========

✅ EXPERIMENT 9 COMPLETED SUCCESSFULLY

==========

📂 Results saved in:
/content/Experiment_09_Results